

7. Statement interface

The **Statement interface** provides methods to execute queries with the database. The statement interface is a factory of ResultSet i.e. it provides factory method to get the object of ResultSet. Use this for general-purpose access to your database. Useful when you are using static SQL statements at runtime. The Statement interface cannot accept parameters.

Commonly used methods of Statement interface:

The important methods of Statement interface are as follows:

- 1) **public ResultSet executeQuery(String sql):** is used to execute SELECT query. It returns the object of ResultSet.
- 2) **public int executeUpdate(String sql):** is used to execute specified query, it may be create, drop, insert, update, delete etc.
- 3) **public boolean execute(String sql):** is used to execute queries that may return multiple results.
- 4) **public int[] executeBatch():** is used to execute batch of commands.

Example of Statement interface

Let's see the simple example of Statement interface to insert, update and delete the record.

```
package com.lokesh;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

public class StatementInterface {
    public static void main(String[] args) {
        try {
            // Load and Register the Driver
            Class.forName("com.mysql.cj.jdbc.Driver");

            // We can also use url like jdbc:mysql://organization
            // If MySQL is running in localhost and default port 3306
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/organization", "root", "root");

            Statement stmt = con.createStatement();
            ResultSet rs = stmt.executeQuery("SELECT * FROM employee");

            System.out.println("Existing records in table");
            while (rs.next()) {
                System.out.println(rs.getInt(1) + "\t" +
                    rs.getString(2) + "\t" + rs.getString(3) + "\t" +
                    rs.getInt(4));
            }

            System.out.println("\nInserting the new record into the Table"
                );
        }
    }
}
```

```
stmt.executeUpdate("INSERT INTO employee values (6, 'Arul',  
                                                    'Senior Developer', 80000)");  
  
System.out.println("\nUpdating the record in table");  
stmt.executeUpdate("UPDATE employee SET salary=85000 WHERE  
                                                    id=6");  
  
System.out.println("\nDeleting the record in table");  
int result = stmt.executeUpdate("delete from employee where  
                                                    id=6");  
  
System.out.println(result + " records affected");  
  
con.close();  
} catch (Exception e) {  
    System.out.println(e);  
}  
}  
}
```

Output:

```
Existing records in table  
1      John Doe      Software Engineer    60000  
2      Jane Smith    Senior Developer     75000  
3      Robert Brown  Project Manager     90000  
4      Emily White   QA Engineer         55000  
5      Michael Green DevOps Engineer    65000
```

```
Inserting the new record into the table
```

```
Updating the record in table
```

```
Deleting the record in table  
1 records affected
```